

# Taller 03 — ETL

Creación e implementación de 5 DAGs en Apache Airflow

Repositorio: Taller-03-ETL

Apache Airflow 3.3.1 · PostgreSQL 16 · Docker Compose

# 1. Introducción y objetivo

Este documento describe el desarrollo del taller práctico de Apache Airflow, cuyo objetivo es diseñar e implementar cinco DAGs (Directed Acyclic Graphs) que automatizan tareas distintas, cada uno con al menos tres tareas interconectadas en una secuencia lógica. Se presenta, para cada DAG, su propósito de negocio, la descripción de cada tarea, el flujo de trabajo y su configuración (paralelismo, dependencias, trigger rules), junto con la evidencia de las pruebas de ejecución realizadas.

El código completo de los cinco DAGs, la infraestructura de despliegue (Docker Compose) y las pruebas automáticas se encuentran en el repositorio que acompaña este documento.

## 2. Arquitectura general

El proyecto sigue el flujo Fuentes de datos → Ingesta/Transformación → Almacenamiento → Orquestación → Análisis, distribuido entre los cinco DAGs. Apache Airflow se despliega en cuatro componentes independientes -scheduler, dag-processor, api-server y triggerer-, todos sobre LocalExecutor, junto a una única instancia de PostgreSQL que cumple dos roles: almacenar los metadatos propios de Airflow (base de datos airflow) y almacenar los datos de negocio que producen los DAGs, en un esquema separado (taller).

```
Fuentes de datos (API REST, archivos)
|
v
Ingesta / Transformacion (tasks de Airflow, Python + pandas)
|
v
PostgreSQL (esquema "taller": tablas de cada DAG)
|
v
Apache Airflow (scheduler + dag-processor + api-server + triggerer)
|
v
UI web (localhost:8080): logs, historial de ejecucion, disparo manual
```

Se descartó sumar Celery, Redis o un segundo motor de base de datos: con cinco DAGs y un volumen de ejecución modesto, LocalExecutor ya ofrece ejecución en paralelo -demostrada explícitamente en el DAG 5- sin las piezas adicionales que implicaría un executor distribuido.

### 3. Stack tecnológico

| Tecnología              | Versión | Propósito                               |
|-------------------------|---------|-----------------------------------------|
| Apache Airflow          | 3.3.1   | Orquestación de los cinco pipelines     |
| PostgreSQL              | 16      | Metadatos de Airflow + datos de negocio |
| Python                  | 3.12    | Lenguaje de los DAGs                    |
| pandas                  | 3.0.5   | Limpieza, transformación y agregación   |
| requests                | 2.34.2  | Consumo de APIs REST                    |
| Docker / Docker Compose | —       | Empaquetado y despliegue                |
| pytest                  | 8.3.4   | Pruebas automáticas de los DAGs         |

### 4. Descripción detallada de los cinco DAGs

Los DAGs están ordenados de menor a mayor complejidad de orquestación: el primero es estrictamente lineal y los siguientes incorporan, uno a uno, XCom intensivo, ramificación condicional (branching), agrupación de tareas (TaskGroup) y ejecución en paralelo con convergencia (fan-in).

#### DAG 1 — weather\_etl\_dag

**Propósito de negocio:** obtener el clima actual de cinco ciudades latinoamericanas desde la API pública Open-Meteo (no requiere API key) y dejarlo disponible en PostgreSQL.

##### Flujo de trabajo:

```
extraer_datos_clima
-> transformar_datos_clima
-> cargar_datos_postgres
-> generar_resumen

(dependencia estrictamente lineal)
```

##### Tareas:

| Tarea                   | Operador       | Qué hace                                                                                                                                         |
|-------------------------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| extraer_datos_clima     | PythonOperator | Consulta la API REST de Open-Meteo para cada ciudad configurada y guarda el JSON crudo en data/raw.                                              |
| transformar_datos_clima | PythonOperator | Convierte el JSON en un DataFrame de pandas y agrega una columna derivada (clasificación frío/templado/cálido). Guarda un CSV en data/processed. |
| cargar_datos_postgres   | PythonOperator | Inserta (upsert) el CSV procesado en la tabla taller.clima_observaciones.                                                                        |
| generar_resumen         | PythonOperator | Consulta la tabla recién cargada y registra un resumen agregado por clasificación de temperatura.                                                |

##### Configuración:

- `schedule="@daily"`, `catchup=False` (evita disparar ejecuciones retroactivas al desplegar).

- retries=2, retry\_delay=5 minutos: una API externa puede fallar de forma transitoria.
- Comunicación entre tareas por XCom (la ruta del archivo generado en un paso es el valor de retorno que recibe el siguiente).

## DAG 2 — sales\_etl\_dag

**Propósito de negocio:** procesar el archivo diario de pedidos de un sistema de ventas, limpiarlo, calcular métricas por categoría/región y cargar detalle y resumen en PostgreSQL.

### Flujo de trabajo:

```
generar_datos_ventas_diarias
-> limpiar_datos_ventas
-> calcular_metricas_ventas
-> cargar_ventas_postgres
-> validar_carga_ventas
```

### Tareas:

| Tarea                        | Operador       | Qué hace                                                                                                          |
|------------------------------|----------------|-------------------------------------------------------------------------------------------------------------------|
| generar_datos_ventas_diarias | PythonOperator | Simula la llegada del archivo diario de pedidos (incluye a propósito nulos, duplicados y montos inválidos).       |
| limpiar_datos_ventas         | PythonOperator | Con pandas: descarta duplicados, nulos en campos clave y montos no positivos.                                     |
| calcular_metricas_ventas     | PythonOperator | Agrupar por categoría y región: total vendido y cantidad de pedidos.                                              |
| cargar_ventas_postgres       | PythonOperator | Carga el detalle limpio y el resumen agregado en PostgreSQL.                                                      |
| validar_carga_ventas         | PythonOperator | Compara filas que se intentó cargar contra filas realmente presentes en la base (control de calidad de la carga). |

### Configuración:

- schedule="@daily", catchup=False.
- Uso intensivo de XCom: cada tarea pasa a la siguiente no solo rutas de archivo sino también métricas (filas originales, filas limpias, filas cargadas).
- La semilla aleatoria de los datos simulados se deriva de la fecha de ejecución (reproducibilidad: la misma fecha siempre genera el mismo dataset).

## DAG 3 — api\_health\_monitor\_dag

**Propósito de negocio:** monitorear la disponibilidad de un conjunto de endpoints HTTP y decidir automáticamente si corresponde generar una alerta.

### Flujo de trabajo:

```
verificar_endpoints
-> evaluar_estado_general (BranchPythonOperator)
-> generar_alerta [solo si algun endpoint fallo]
-> registrar_estado_ok [solo si todos respondieron bien]
-> generar_reporte_monitoreo [siempre, sea cual sea la rama ejecutada]
```

### Tareas:

| Tarea                     | Operador             | Qué hace                                                                                                                                                                         |
|---------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| verificar_endpoints       | PythonOperator       | Consulta cada endpoint configurado y registra código de estado y latencia. No lanza excepción si un endpoint falla: es un resultado de negocio válido, no un error de ejecución. |
| evaluar_estado_general    | BranchPythonOperator | Decide, según los resultados anteriores, cuál de las dos ramas siguientes ejecutar.                                                                                              |
| generar_alerta            | PythonOperator       | Rama ejecutada solo si algún endpoint no respondió correctamente; registra el detalle.                                                                                           |
| registrar_estado_ok       | PythonOperator       | Rama ejecutada solo si todos los endpoints respondieron correctamente.                                                                                                           |
| generar_reporte_monitoreo | PythonOperator       | Se ejecuta siempre (trigger_rule=NONE_FAILED_MIN_ONE_SUCCESS) y persiste el resultado en PostgreSQL.                                                                             |

### Configuración:

- Único DAG con lógica condicional (branching) y trigger rules explícitas.
- schedule="\*/30 \* \* \* \*" (cada 30 minutos, cadencia típica de un monitor).
- Endpoints configurables vía la variable de entorno TALLER\_MONITOR\_ENDPOINTS; el valor por defecto incluye un endpoint que siempre responde con error, para que la rama de alerta pueda observarse sin depender de que un servicio real esté caído en ese momento.

## DAG 4 — data\_quality\_ingestion\_dag

**Propósito de negocio:** ingerir un archivo enviado por un socio externo, validar su calidad y clasificarlo automáticamente como aceptado o rechazado.

### Flujo de trabajo:

```
generar_archivo_entrada
-> TaskGroup "validaciones_calidad" (en paralelo):
  validar_esquema
  validar_valores_nulos
  validar_duplicados
-> consolidar_resultado_validacion
-> clasificar_y_mover_archivo
-> generar_reporte_calidad
```

## Tareas:

| Tarea                                                              | Operador                      | Qué hace                                                                                                            |
|--------------------------------------------------------------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------|
| generar_archivo_entrada                                            | PythonOperator                | Simula la llegada de un archivo con problemas de calidad deliberados (nulos y un registro duplicado).               |
| validar_esquema /<br>validar_valores_nulos /<br>validar_duplicados | PythonOperator<br>(TaskGroup) | Tres validaciones independientes entre sí; al no depender una de otra, Airflow las ejecuta en paralelo.             |
| consolidar_resultado_vali<br>dacion                                | PythonOperator                | Reúne el resultado de las tres validaciones (vía XCom, con el prefijo del TaskGroup) y decide el veredicto general. |
| clasificar_y_mover_archiv<br>o                                     | PythonOperator                | Mueve el archivo a data/processed (aprobado) o data/quarantine (rechazado) y registra el resultado en PostgreSQL.   |
| generar_reporte_calidad                                            | PythonOperator                | Deja un resumen legible del resultado de calidad en el log.                                                         |

## Configuración:

- Único DAG con TaskGroup.
- schedule=None (disparo manual/por evento): representa un pipeline que en producción se activaría cuando el archivo efectivamente llega, no en un horario fijo.
- Una validación que detecta un problema no hace fallar su tarea: para calidad de datos, "el archivo tiene 2 nulos" es un resultado de negocio válido que se registra y se actúa en consecuencia.

## DAG 5 — backup\_cleanup\_dag

**Propósito de negocio:** respaldar las tablas clave del esquema taller, verificar la integridad del backup generado y aplicar una política de retención.

### Flujo de trabajo:

Rama A: crear\_backup\_tablas -> verificar\_integridad\_backup

Rama B: limpiar\_backups\_antiguos (en paralelo; no depende de la rama A)

Ambas ramas confluyen (fan-in) en:

verificar\_espacio\_almacenamiento -> reportar\_estado\_mantenimiento

### Tareas:

| Tarea                            | Operador       | Qué hace                                                                                                                                                                                                 |
|----------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| crear_backup_tablas              | PythonOperator | Exporta las tablas configuradas a CSV y las empaqueta en un .tar.gz con timestamp.                                                                                                                       |
| verificar_integridad_backup      | PythonOperator | Confirma que el archivo existe, pesa más de 0 bytes y es un .tar.gz legible. A diferencia del DAG 4, aquí un resultado negativo sí hace fallar la tarea: un backup corrupto es una falla operativa real. |
| limpiar_backups_antiguos         | PythonOperator | No depende de las tareas anteriores -corre en paralelo- y elimina backups más antiguos que la política de retención (Airflow Variable, 7 días por defecto).                                              |
| verificar_espacio_almacenamiento | BashOperator   | Espera a que terminen integridad y limpieza (fan-in) y reporta con du -sh el espacio usado por los backups.                                                                                              |
| reportar_estado_mantenimiento    | PythonOperator | Consolida backup, integridad y limpieza en la tabla taller.mantenimiento_backups.                                                                                                                        |

### Configuración:

- Único DAG con ejecución en paralelo con convergencia (fan-in): limpiar\_backups\_antiguos corre al mismo tiempo que crear\_backup\_tablas/verificar\_integridad\_backup, y ambas ramas confluyen en verificar\_espacio\_almacenamiento.
- Combina PythonOperator con BashOperator y con una Airflow Variable (taller\_backup\_retention\_days).
- schedule="0 2 \* \* \*" (2 AM, horario típico de mantenimiento).

## 5. Pruebas y evidencia de ejecución

Antes de esta entrega, los cinco DAGs se ejecutaron de extremo a extremo con `airflow dags test` (el comando de Airflow que corre un DAG completo de forma sincrónica, tarea por tarea, contra una base de datos real) sobre Apache Airflow 3.3.1 y PostgreSQL 16. La siguiente tabla resume el resultado obtenido en esa validación.

| DAG                                | Resultado                    | Detalle                                                                                                                                                                                                                                                                                            |
|------------------------------------|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DAG 1 · weather_etl_dag            | Validado por partes          | Transformación, carga (upsert) y resumen probados contra PostgreSQL real con datos de muestra. La llamada a la API de Open-Meteo no pudo probarse en el entorno de validación por restricciones de red de ese entorno, pero sigue el esquema de respuesta documentado oficialmente por Open-Meteo. |
| DAG 2 · sales_etl_dag              | state=success                | Las 5 tareas finalizaron correctamente. La limpieza detectó y descartó exactamente los 17 registros "sucios" inyectados de prueba (8 nulos, 5 duplicados, 4 montos inválidos).                                                                                                                     |
| DAG 3 · api_health_monitor_dag     | state=success (dos corridas) | Se validaron ambas ramas del branching por separado: una corrida con todos los endpoints saludables (rama registrar_estado_ok) y otra con un endpoint caído a propósito (rama generar_alerta).                                                                                                     |
| DAG 4 · data_quality_ingestion_dag | state=success                | Las 7 tareas (incluidas las 3 del TaskGroup, en paralelo) finalizaron correctamente. La validación detectó los 2 nulos y el duplicado inyectados de prueba y clasificó el archivo en quarantine, como se esperaba.                                                                                 |
| DAG 5 · backup_cleanup_dag         | state=success                | Las 5 tareas finalizaron correctamente, incluida la convergencia (fan-in) entre la verificación de integridad y la limpieza en paralelo.                                                                                                                                                           |

### Capturas de pantalla de la UI

Este documento se generó junto con el código, antes de desplegar el entorno con `docker compose up -d`. Las capturas de pantalla de la interfaz web de Airflow -que exige el enunciado del taller como evidencia adicional- deben tomarse una vez levantado ese entorno y se anexan por separado (carpeta docs/capturas del repositorio). La siguiente checklist indica qué capturar para dejar evidencia completa de cada DAG:

| # | Captura                             | Qué debe mostrar                                                                                            |
|---|-------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1 | Vista "DAGs"                        | Listado completo mostrando los 5 DAGs activos, sin errores de importación.                                  |
| 2 | Vista "Graph" (por cada DAG)        | El grafo de tareas y sus dependencias, tal como se describe en la sección 4.                                |
| 3 | Vista "Grid" tras disparar cada DAG | El historial de ejecución con las tareas en success (verde); en el DAG 3, una rama debe verse skipped.      |
| 4 | Logs de al menos una tarea por DAG  | El detalle de una tarea (por ejemplo, cargar_datos_postgres o generar_reporte_calidad) mostrando su salida. |

Además de las pruebas manuales por UI, el repositorio incluye una suite automática de pytest (`tests/test_dag_logic.py`) que valida la lógica pura de negocio y la estructura de los cinco DAGs (cantidad mínima de tareas, dependencias del branching y del fan-in).

## 6. Errores comunes detectados durante el desarrollo

- **Variable.get() got an unexpected keyword argument 'default\_var'.** Airflow 3 introdujo un nuevo Task SDK (airflow.sdk); su clase Variable cambió la firma de .get(): el parámetro pasó a llamarse default en lugar de default\_var. Se corrigió en el DAG 5.
- **Un endpoint externo responde 403 Forbidden sin motivo aparente.** Varias APIs (GitHub incluida) rechazan solicitudes sin cabecera User-Agent. El DAG 3 la incluye explícitamente.
- **Un DAG nuevo no aparece en la lista de DAGs ni puede probarse por CLI.** En Airflow 3, el DAG Processor -componente separado del scheduler- es quien parsea los archivos de dags/ y los registra; la UI y la CLI leen desde ahí. Con docker compose esto ocurre de forma continua y automática; al probar por CLI de forma manual puede requerir airflow dags reserialize.
- **Archivos generados dentro de Docker con dueño root en el host (Linux).** Se resuelve definiendo AIRFLOW\_UID en .env con el resultado de id -u.
- **Una tarea parece "colgada" varios minutos después de fallar.** Si falla y el DAG tiene retries configurados, Airflow espera el retry\_delay (5 minutos en la mayoría de los DAGs de este taller) antes de reintentar; no es que el sistema esté congelado.
- **ModuleNotFoundError: pandas / requests / psycpg2.** La imagen base de Airflow no incluye estas librerías; se resuelve extendiéndola con un Dockerfile propio que instala requirements.txt.
- **ImportError al importar PostgresHook.** El provider de PostgreSQL (apache-airflow-providers-postgres) no viene instalado por defecto y debe agregarse explícitamente.

## 7. Conclusión

El taller quedó implementado con cinco DAGs funcionales, cada uno con al menos tres tareas interconectadas en una secuencia lógica y con un propósito de negocio distinto: extracción desde una API REST, limpieza y agregación con pandas, monitoreo con lógica condicional, validación de calidad de datos con TaskGroups y mantenimiento con ejecución en paralelo. El código, la infraestructura de despliegue y las pruebas automáticas se encuentran en el repositorio adjunto.